Model-Driven Automation of Cyber-Physical Systems via AADL and LLMs

Abdul Wahab
Ontario Tech University
Oshawa, Ontario, Canada
abdul.wahab2@ontariotechu.net

Akramul Azim
Ontario Tech University
Oshawa, Ontario, Canada
akramul.azim@ontariotechu.ca

Abstract

The development of cyber-physical systems presents fundamental challenges in system design, verification, and implementation, particularly in safety-critical domains where reliability and real-time performance are paramount. Classical development approaches rely heavily on manual coding and modeling, which are resource-intensive, error-prone, and require substantial domain expertise. While Model-Driven Engineering (MDE) methodologies, such as the Architecture Analysis and Design Language (AADL), provide systematic abstraction tools, they struggle with the dynamic and unpredictable nature of CPS environments. To address these challenges, we propose an integrated method that combines AADL automation with code generation using large language models (LLMs) to automate model generation, intelligent code synthesis, and error correction. Our approach uses a dual-phase approach: first, automating AADL model generation from natural language requirements using domain-specific LLMs, and second, implementing intelligent code generation with Chain-of-Thought (CoT) prompting for automated error detection and correction. The framework integrates formal verification techniques, runtime assurance methods, and version control to ensure safety-critical compliance while reducing development time and manual effort. We validate our approach through comprehensive case studies including unmanned aerial vehicles and smart building systems, showing improvements in development efficiency, code quality, and system reliability. Our results show an average 70% reduction in development time, syntax accuracy exceeding 88%, and a 25% reduction in energy consumption for deployed systems.

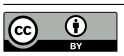
CCS Concepts

• **Software and its engineering → Model-driven software engineering**; • **Computer systems organization → Embedded and cyber-physical systems**.

Keywords

Cyber-Physical Systems, AADL, Model-Driven Engineering, LLMs, Code Generation, Automated Verification

ACM Reference Format:
Abdul Wahab and Akramul Azim. 2026. Model-Driven Automation of Cyber-Physical Systems via AADL and LLMs. In *22nd International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE '26), July 5, 2026, Montreal, QC, Canada.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3803846.3807460



This work is licensed under a Creative Commons Attribution 4.0 International License.
PROMISE '26, Montreal, QC, Canada
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2584-5/26/07
https://doi.org/10.1145/3803846.3807460

1 INTRODUCTION

The rise of cyber-physical systems represents a fundamental paradigm shift in system engineering, introducing unprecedented complexity in the integration of computational processes with physical components through feedback control loops and communication networks. These systems, characterized by tight coupling between software, hardware, and physical processes, have become ubiquitous in modern society, powering critical infrastructure including autonomous vehicles, smart grids, medical devices, aerospace systems, and industrial automation. The continuous interaction between discrete computational elements and continuous physical dynamics creates unique challenges that traditional engineering approaches struggle to address effectively [3, 27].

In safety-critical domains, where system failures can result in catastrophic consequences including loss of life, property damage, or environmental harm, the verification and validation of CPS behavior becomes not merely important but absolutely essential. These systems must maintain stringent safety standards while operating in real-time within distributed, often unpredictable environments. The core engineering challenges stem from several interrelated factors: continuous-discrete interactions that require hybrid system modeling techniques, complex temporal dependencies that demand precise timing analysis, concurrent execution patterns that necessitate careful synchronization, and the imperative to ensure both functional correctness and non-functional properties such as performance, reliability, and security.

Recent advances in system engineering methodologies have attempted to address these challenges through different approaches, with Model-Driven Engineering emerging as a particularly promising paradigm. MDE offers systematic abstraction methods through domain-specific modeling languages, helping development through automated model transformations, documentation generation, code synthesis, and systematic verification approaches [19, 45]. However, despite these advances, MDE faces limitations. The manual development of system models remains time-consuming and resource-intensive, requiring substantial domain expertise throughout the development lifecycle. Furthermore, while MDE provides valuable abstractions, ensuring the accuracy and completeness of these abstractions for safety-critical CPS applications remains challenging, particularly when dealing with complex real-world scenarios involving uncertain environments and evolving requirements [14, 24].

The AADL represents an advancement within the MDE paradigm, specifically tailored for safety-critical cyber-physical systems. Originally developed by the Society of Automotive Engineers (SAE), AADL has established itself as a comprehensive framework for

systematic modeling and analysis of complex embedded systems. The language provides standardized methods for representing both hardware and software architectures, enabling precise specification of system components, their interactions, timing constraints, and resource requirements. In safety-critical domains, AADL offers robust methods for addressing fundamental design concerns through sophisticated safety analysis capabilities, comprehensive security specification and verification support, and automated code generation features that improve consistency between system requirements and implementations [17].

Despite the considerable advantages of AADL in safety-critical systems development, several persistent challenges limit its widespread adoption and effectiveness. First, the manual development of AADL models is labor-intensive and requires continuous involvement of domain experts, creating bottlenecks in the development process and increasing overall costs. Second, while AADL supports multiple representation formats including graphical, textual, and XML-based approaches, existing methodologies predominantly rely on graphical representations to mitigate human coding errors, potentially sacrificing benefits such as detailed specifications, precise control, efficient version management, and seamless integration with modern software development workflows. Third, the high dependency on domain expertise throughout the development lifecycle limits scalability and accessibility of AADL-based approaches.

Concurrently, the rapid evolution of artificial intelligence, particularly the development of LLMs trained on vast textual datasets, has opened new possibilities for automating complex software engineering tasks. LLMs such as GPT-4, Claude, and Llama have demonstrated remarkable capabilities in understanding and generating structured technical language, code synthesis across multiple programming languages, and reasoning about complex system architectures [5, 6]. These models have shown promising results in general software development contexts, including code completion, bug detection, and documentation generation. However, their application to cyber-physical systems development, which requires deep understanding of hardware-software interactions, real-time constraints, safety properties, and physical dynamics, remains largely unexplored.

Furthermore, existing code generation tools and methodologies face lots of challenges when applied to CPS development. Natural language ambiguity introduces difficulties in translating high-level specifications into precise, executable code. Traditional MDD practices lack the semantic depth necessary to capture intricate real-world scenarios, and the transformation of models into executable code typically involves extensive manual effort in developing and maintaining code generators. Current automated tools struggle with the complex requirements of CPS environments, including real-time performance constraints, safety-critical requirements, hardware platform dependencies, and the need for formal verification of generated artifacts.

To address the above mentioned challenges, we present an integrated technique that mutually combines AADL automation with AI-enhanced code generation capabilities. Our approach uses domain-specific LLMs to automate the entire CPS development pipeline, from initial requirements specification through model generation, code synthesis, verification, and deployment. The framework presents intelligent assistance methods that reduce dependency on domain experts and maintain the high quality and safety assurance levels required for CPS applications. By integrating formal verification methods, automated error correction with Chain-of-Thought reasoning, and runtime assurance methods, our methodology provides a solution that addresses both the modeling and implementation challenges in modern CPS development. Our primary contributions are as follows:

- We introduce a unified framework that seamlessly integrates AADL-based model-driven engineering with LLM-assisted synthesis, enabling automated translation from natural language requirements to CPS design artifacts.
- We develop a reasoning-driven methodology for automated model derivation and code synthesis, ensuring architectural consistency, safety, and CPS-specific constraints through domain-aware LLM reasoning and CoT prompting.
- We propose an end-to-end verification and adaptive feedback architecture, combining formal methods and runtime assurance to guarantee correctness and support continuous evolution across diverse CPS domains.

The remainder of the paper is organized as follows: Section 2 reviews related work, Section 3 presents the problem statement, Section 4 describes the framework design and AADL automation pipeline, Section 5 provides case studies, Section 6 discusses the approach and its limitations, and Section 7 concludes the paper.

2 RELATED WORK

2.1 Cyber-Physical Systems Development

Cyber-physical systems represent complex, integrated architectures that seamlessly combine computational elements with physical processes, creating bidirectional interaction between the cyber and physical domains [27]. These systems aim to enhance capabilities of large-scale infrastructure including smart grids [44], autonomous vehicles [15], healthcare systems [13, 46], and industrial automation [34]. The architecture of CPS is inherently complex, requiring simultaneous management of dynamic physical processes and the computational systems that monitor and control them. This complexity necessitates robust and flexible System Development Life Cycle methodologies capable of managing both cyber and physical components while ensuring reliability, safety, and real-time performance [3, 11, 17].

CPS development traditionally employs conventional SDLC approaches including Waterfall, Agile, and the V-Model, each offering distinct advantages and facing unique challenges. The Waterfall model's linear, sequential structure facilitates documentation and phase-by-phase management but proves rigid when addressing the iterative modifications frequently required in CPS projects [25]. Conversely, Agile methodologies such as Scrum, Kanban, and Extreme Programming offer iterative approaches enabling increased collaboration, flexibility, and continuous feedback. However, when applied to CPS, Agile faces challenges in maintaining synchronization between software and hardware components and achieving strict safety and real-time performance requirements [45]. The iterative nature of Agile sometimes conflicts with the detailed planning required for physical components, which often have long lead times for testing and development.

Specialized models such as the Spiral and Iterative approaches address some limitations through cyclic development methods and continuous feedback-driven improvement. These models prove particularly valuable for high-risk CPS projects where failure could result in serious safety and financial consequences. However, they are often costly and difficult to deploy, especially for smaller initiatives [7]. The V-Model, extending the Waterfall approach by matching each development step with corresponding testing phases, suits safety-critical CPS applications but lacks flexibility to adapt to dynamic, evolving requirements frequently encountered in CPS development.

Despite the variety of available SDLC models, many approaches are inadequately equipped to handle the dual requirements of CPS, where both cyber and physical components must be developed and tested concurrently. The unique characteristics of CPS projects, including requirements for real-time interaction with physical environments and extensive testing across hardware and software domains, highlight limitations of current methodologies [33]. Recent studies indicate that CPS development faces persistent challenges including sustainability, integration across cyber and physical layers, and testing frameworks addressing both real-time and safety-critical aspects [1, 26, 38].

An emerging trend in the literature emphasizes the increasing importance of sustainability in CPS development. Sustainable CPS design focuses on technical and economic robustness, ensuring systems are adaptable, energy-efficient, and maintainable throughout their lifecycle [19, 36, 45]. This requires architectural designs capable of evolving as requirements emerge while maintaining system integrity and minimizing operational costs. The design of sustainable CPS is further complicated by needs for self-adaptation methods that adjust behavior in response to environmental changes [12]. These methods often rely on feedback loops such as the Monitor-Analyze-Plan-Execute over shared Knowledge (MAPE-K) architecture, widely adopted in CPS to manage real-time decision-making and system adjustments [26, 33].

2.2 Model-Driven Engineering and AADL

Model-Driven Engineering has emerged as an effective approach for managing the complexity of software-intensive systems by providing abstraction through domain-specific modeling languages. It supports development through model transformations, automated generation of documentation, code, and verification artifacts while promoting systematic verification. The AADL, developed by the Society of Automotive Engineers (SAE), extends these capabilities for safety-critical cyber-physical systems by providing standardized methods to model and analyze both hardware and software architectures.

In safety-critical domains, AADL offers robust methods to address key design concerns, including safety analysis through hazard assessment and mitigation, as well as security specification and verification throughout the development lifecycle. It also supports automated code generation, improving consistency between system requirements and implementation while reducing errors and development time. Additionally, AADL provides analytical capabilities for performance optimization, timing analysis, and resource allocation verification, which are important for reliable safety-critical system development. The significance of AADL lies primarily in its support for cross-domain integration and system-level analysis, enabling developers to reason about complex architectural decisions and their implications across various engineering disciplines. This proves particularly valuable in contexts demanding rigorous certification compliance where systematic assessment of safety features is mandatory. Furthermore, the framework promotes direct interaction among stakeholders, improving communication throughout the development process.

Despite considerable advantages, AADL faces several persistent challenges. Manual development of AADL models remains time-consuming and resource-intensive, requiring substantial domain expertise throughout the development lifecycle. While AADL supports multiple representation formats including graphical, textual, and XML, existing approaches predominantly rely on graphical representations to mitigate human coding errors, potentially sacrificing benefits of textual representation such as detailed specifications, precise control, and efficient version management. The high dependency on domain experts creates bottlenecks and increases development costs.

2.3 LLMs in Software Engineering

Recent advances in deep learning, particularly the development of LLMs, have created new opportunities for optimizing software development processes. LLMs, built upon transformer-based neural architectures and trained on vast internet-scale text datasets, have demonstrated exceptional capabilities in understanding and generating structured technical content [6, 39]. These models exhibit impressive performance in logic-based reasoning, complex problem-solving, and various language-task benchmarks, generating text often indistinguishable from human-written content.

The applications of LLMs span diverse domains, showing effectiveness as reliable foundational models in specialized areas including consumer health, medicine, education, finance, chip design, and environmental science [21, 22, 35]. The breadth of their applicability underscores significant potential for interdisciplinary research and application development. Research has shown that LLMs can generate software with remarkable proficiency, rivaling output of skilled developers across multiple programming languages [9, 20, 28].

To enhance LLM effectiveness, researchers have explored integration with external tools and APIs. Studies demonstrate that limitations in specific tasks, such as arithmetic calculations, can be mitigated through API access [6]. Recent research has shown utility of fine-tuning to improve interaction between LLMs and tooling APIs [31]. However, fully harnessing LLM capabilities requires high-quality prompting to receive consistently valuable outputs. Various studies have developed frameworks and workflows to optimize LLM responses [40–43], often involving detailed and unambiguous prompts, additional context provision, and self-evaluation methods.

Research on improving code generation capabilities has explored various techniques including model fine-tuning for specific programming languages [8, 29] and creating methodologies for better prompting, such as self-debugging [10, 37], task decomposition [18], and prompting frameworks [32]. While previous studies have evaluated LLM-based code generation for multiple programming

languages including C/C++ commonly used in CPS development, explicit exploration of LLMs in CPS contexts remains limited.

Existing research on hardware-level code generation has primarily focused on developing tools and frameworks simplifying hardware-level programming for novices without using LLMs [2, 4, 30]. Other work has focused on generating hardware-level programs, often in hardware description languages, from higher-level languages or design representations [16, 23]. Although some tools and resources exist for LLM-based CPS code generation, comprehensive evaluation of code generation for CPS development remains lacking.

2.4 Research Gaps and Motivation

Despite notable advances in both model-driven engineering and artificial intelligence, several gaps persist in the intersection of these domains for cyber-physical systems development. First, while AADL provides powerful modeling capabilities, the manual effort required for model creation remains a bottleneck. Second, although LLMs demonstrate impressive code generation capabilities in general software contexts, their application to safety-critical CPS with stringent real-time, reliability, and verification requirements remains largely unexplored. Third, existing approaches lack integration between high-level architectural modeling and low-level code generation, creating gaps in traceability and consistency. Fourth, automated error detection and correction mechanisms specifically designed for CPS contexts, considering both functional and non-functional requirements, are absent from current methodologies.

Our work addresses these gaps by introducing an integrated approach that combines strengths of AADL-based modeling with capabilities of domain-adapted LLMs, creating a viable solution for automated, verifiable, and efficient cyber-physical systems development.

3 PROBLEM STATEMENT

The development of CPS presents multifaceted challenges spanning modeling, code generation, verification, and deployment, which current approaches struggle to address. General-purpose modeling languages, such as UML, lack the semantic richness to capture CPS-specific dynamics, timing constraints, and domain knowledge. As a result, models may accurately represent structural and behavioral aspects yet omit critical information required for safety-critical systems. Transforming these models into executable code demands extensive manual development and maintenance of code generators, limiting adaptability across programming languages and platforms, with tools such as Eclipse Papyrus, MagicDraw, and IBM Rational Rhapsody offering restricted flexibility. AADL mitigates some limitations by providing embedded-system-specific constructs; however, manual model development remains time- and resource-intensive, creating bottlenecks. Graphical representations, while reducing coding errors, compromise textual benefits, including precise specifications, version control, collaborative merging, and integration with continuous development pipelines. Automated code generation is further challenged by natural language ambiguities and limited CPS-specific knowledge in existing models, often yielding incomplete or incorrect code requiring extensive review. Limited traceability between high-level models and low-level implementations complicates verification and certification. Additionally, CPS operate under stringent real-time and resource constraints, which many automated tools fail to enforce, producing artifacts that are functionally correct but practically undeployable. The integration of formal verification and runtime assurance remains insufficient, further impeding reliable CPS development.

3.1 Research Objectives

Given these challenges, our work aims to develop an integrated technique that addresses the full spectrum of CPS development concerns. Specifically, we seek to:

(1) Automate AADL model generation from natural language requirements while maintaining semantic accuracy and completeness. (2) Enable intelligent code generation that considers both functional requirements and CPS-specific constraints. (3) Integrate formal verification techniques throughout the development pipeline. (4) Implement automated error detection and correction methods designed to CPS contexts. (5) Establish seamless traceability from requirements through models to verified implementations. (6) Reduce dependency on continuous domain expert involvement while maintaining quality and safety assurance. (7) Demonstrate practical viability through comprehensive case studies in diverse CPS domains.

The following section presents our proposed methodology addressing these objectives through an integrated framework combining AADL automation with LLMs-based code generation capabilities.

4 Framework Design and Architecture

We propose a framework that integrates AADL-based model-driven engineering with LLM-powered code generation, forming a unified pipeline for cyber-physical systems development. As shown in Figure 1, it combines architectural modeling and implementation with safety-critical compliance via integrated verification. Our method extend the Double V-Model, the approach uses automation, formal methods, and adaptive feedback across interconnected phases: requirements processing, AADL model generation, code synthesis, verification and validation, deployment, and operational feedback. Through progressive refinement, artifacts undergo multiple validation stages, enabling early error detection and reducing late-stage correction costs. Full traceability ensures audit trails, supporting rigorous safety certification.

4.1 Phase 1: Requirements Processing and AADL Model Generation

4.1.1 Natural Language Requirements Processing. The development process commences with unstructured natural language specifications, which may originate from stakeholder reports, regulatory guidelines, or system requirements documents. These inputs, often containing ambiguous or domain-specific terminology, are processed by specially fine-tuned LLMs trained on expert-developed corpora from safety-critical domains including avionics (DO-178C), automotive (ISO 26262), and systems engineering (ARP4754A).

The LLM-based requirements processor employs a multi-stage analysis pipeline. First, it performs linguistic analysis to identify

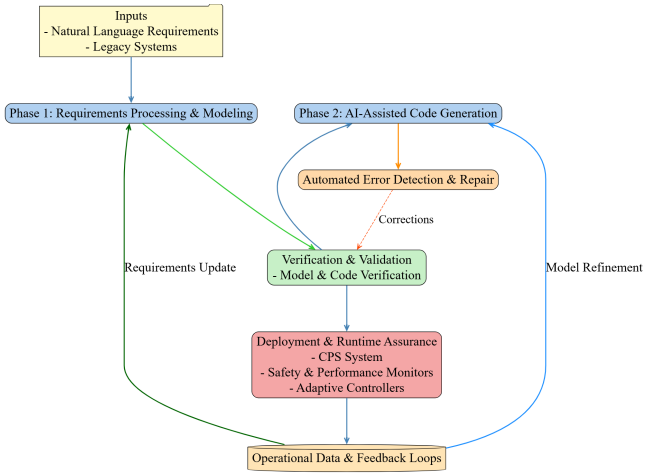


Figure 1: Proposed framework architecture illustrating the complete pipeline from requirements specification through AADL model generation, LLM-driven code synthesis, verification, and deployment, with dual feedback loops enabling incremental refinement and cyclic improvement.

key system entities, functional requirements, non-functional constraints, and relationships among components. Second, it resolves ambiguities through contextual reasoning and, when necessary, generates clarification queries for human review. Third, it structures the processed requirements into a machine-readable intermediate representation using JSON format. This structured representation captures essential system properties including functional descriptions, safety constraints, performance criteria, timing requirements, hardware specifications, and communication patterns.

The transformation from natural language input NL_{input} to structured requirements R_{struct} is formalized as:

$$R_{struct} = \tau_{req}(NL_{input}, K_{domain}) \tag{1}$$

where τ_{req} represents the requirements transformation function and K_{domain} denotes the domain-specific knowledge base incorporated during LLM fine-tuning.

4.1.2 AADL Model Synthesis. Following requirements structuring, proprietary Python scripts interact with the Open Source AADL Tool Environment (OSATE2) to generate comprehensive AADL architectural models. This conversion proceeds through two major steps:

Component Synthesis: The structured requirements are analyzed to identify system components and their characteristics. Functional and non-functional properties are mapped to appropriate AADL constructs including `system`, `process`, `thread`, `processor`, `memory`, `bus`, and `device`. Each component is instantiated with properties derived from the requirements, such as execution time, period, deadline, memory size, and scheduling protocols.

Interaction Mapping: The communication structure is defined in terms of control flows and data dependencies, represented as AADL `ports`, `connections`, and `subprogram calls`. Our approach automatically infers connection types (data, event, or event-data) based on the nature of information exchange specified in requirements. Interface consistency is verified to ensure that connected ports have compatible types and that all required connections are established.

The AADL generation process can be expressed formally as:

$$M_{AADL} = \langle C, I, P, T \rangle = \sigma(R_{struct}) \tag{2}$$

where C represents the set of components, I the set of interfaces and connections, P the set of properties, T the set of timing constraints, and σ is the synthesis function.

4.1.3 Metamodel Construction for Enhanced Code Generation. In parallel with AADL model generation, the framework constructs a comprehensive metamodel that serves as input for the AI-enhanced code generation phase. This metamodel $M = \langle S, B, C, T \rangle$ consists of four interconnected layers:

Structural Layer (S): Defines static system elements including components, interfaces, dependencies, and their relationships using formal component modeling techniques.

Behavioral Layer (B): Captures dynamic interactions and processes through hybrid automata and temporal specifications modeling both discrete and continuous system behaviors.

Constraint Layer (C): Encodes safety properties, timing constraints, resource limitations, and operational boundaries critical for CPS reliability and safety.

Timing Layer (T): Specifies real-time requirements, deadline constraints, and temporal dependencies governing system responsiveness and performance.

The metamodel ensures overall model integrity by specifying rules and constraints governing system architecture while preserving traceability from requirements to implementation.

4.2 Phase 2: LLM-based Code Generation with Error Correction

4.2.1 Domain-Adapted Code Generation. The code generation phase employs fine-tuned LLM architectures specifically adapted for CPS development. The generation process begins with construction of sophisticated domain-specific prompts that embed metamodel information, AADL architectural specifications, target platform constraints, safety properties expressed in Computation Tree Logic and Linear Temporal Logic, and relevant examples from a curated knowledge base. The prompting strategy uses few key elements:

Template-Based Structures: Common CPS patterns are encoded as reusable templates, accelerating generation while ensuring consistency with best practices.

Safety Property Embedding: Critical safety requirements are directly embedded in prompts, including properties of the form $\Box\neg\phi$ (safety properties ensuring bad states are never reached) and $\phi \rightarrow \Diamond_{\leq t}\psi$ (bounded liveness properties ensuring timely responses).

Platform-Specific Adaptations: Hardware and runtime environment characteristics are incorporated to ensure generated code is compatible with target platforms.

Few-Shot Learning: Examples from validated CPS implementations provide concrete reference points, improving generation quality. The code generation function can be expressed as:

$$C_{gen} = \gamma(M, M_{AADL}, K_{pool}, \Phi_{safety}) \quad (3)$$

where γ represents the generation function, K_{pool} is the knowledge pool of validated implementations, and Φ_{safety} encodes safety properties.

4.2.2 Chain-of-Thought Based Error Detection and Correction. A distinctive feature of our framework is the integration of automated error detection and correction using CoT reasoning. When the generated code undergoes initial validation and issues are detected, the system uses structured analytical approaches to diagnose and resolve problems. The error correction process operates as follows:

Error Analysis: Detected errors are categorized by type (syntactic, semantic, timing violations, safety property violations, resource constraint violations) and analyzed to determine root causes.

CoT Reasoning: The LLM generates step-by-step reasoning chains that explore potential fixes, considering both immediate corrections and broader implications for system behavior, performance, and safety.

Solution Generation: Based on the reasoning chains, candidate solutions are synthesized. Multiple alternatives may be generated for complex issues.

Solution Validation: Each candidate solution undergoes rigorous testing including compilation checks, unit testing, timing analysis, safety property verification, and resource usage validation. Solutions failing validation are discarded.

Knowledge Pool Update: Successfully validated solutions are added to the knowledge pool, enriching the system's capability for handling similar issues in future:

$$K_{pool}^{t+1} = K_{pool}^{t} \cup \{(P_{new}, C_{verified}, R_{passed})\} \quad (4)$$

where P_{new} represents the problem, $C_{verified}$ the verified solution, and R_{passed} the validation results.

Iterative Refinement: If initial correction attempts fail, the system uses iterative prompting, incorporating detailed failure information in subsequent attempts. This continues for a maximum of five iterations to ensure termination while providing multiple opportunities for successful correction.

4.3 Formal Verification Integration

4.3.1 Model-Level Verification. The generated AADL models undergo comprehensive formal verification using integrated analysis tools within OSATE2. Syntactic verification ensures that all AADL constructs conform to language specifications. Interface consistency checking validates that connected components have compatible port types and that all required connections are properly established. Model completeness analysis verifies that all system components are properly defined and integrated.

Beyond structural checks, our system uses the AGREE (Assume Guarantee REasoning Environment) tool for component-level contract specification and verification. These contracts specify assumptions and guarantees of system behavior, including safety invariants, timing constraints, and functional properties. Contracts are formally verified using model-checking techniques to ensure component-level conformance with critical system properties.

4.3.2 Code-Level Verification. Generated code undergoes immediate formal verification through automatic transformation into verification models. The framework employs two complementary verification approaches:

Bounded Model Checking: CBMC (C Bounded Model Checker) is used to verify safety properties of generated C/C++ code. This includes checking for common programming errors such as buffer overflows, null pointer dereferences, and integer overflows, as well as domain-specific safety properties derived from requirements.

Timed Automata Verification: For real-time constraints, the system transforms code into UPPAAL timed automata models and verifies temporal properties. This ensures that generated implementations satisfy specified timing requirements and deadlines.

The verification outcomes $V(C_{gen}) = \{v_1, v_2, ..., v_n\}$ represent results of comprehensive property checking. This integration of formal verification during code generation, rather than as a post-processing step, enables early detection of specification violations and guides the generation process toward safer implementations.

4.4 Runtime Assurance System

To ensure continued safety during system operation, the framework incorporates a runtime assurance system providing three complementary techniques:

Safety Monitors: Continuously check safety invariants during execution, ensuring the system never violates critical safety properties. When violations are detected or predicted, monitors trigger appropriate responses including warnings, graceful degradation, or emergency shutdown.

Performance Monitors: Track timing constraints and resource usage in real-time, providing early warning of potential violations. This enables proactive corrective actions before actual deadline misses or resource exhaustion occurs.

Adaptive Controllers: Implement fallback methods when violations are detected, maintaining system safety through graceful degradation or emergency procedures. The runtime assurance system maintains a safety envelope $S_e = \{s \in \mathcal{S} | \phi_{safety}(s) = true\}$ and employs sophisticated control approaches ensuring the system state never exits this envelope.

4.5 Dual-Loop Feedback Method

A distinctive feature of our methodology is its dual-loop feedback method supporting both incremental and cyclic system improvement:

Incremental Feedback Loop: Operates during early development phases, facilitating stepwise refinement of system models and requirements. When ambiguous specifications are detected (e.g., "respond quickly"), validators or the LLM generate prompting clarification questions calling for explicit definitions (e.g., millisecond thresholds for latency). This interactive feedback accelerates requirement clarification and model alignment.

Cyclic Feedback Loop: Engages during system runtime, collecting performance logs, fault reports, and environmental updates. This operational data is analyzed and used to update models and requirement specifications. These updates are managed through the same pipeline, allowing continuous learning from real-world conditions and evolving threats.

4.6 Version Control and Traceability

To ensure traceability and robust change management, the framework integrates Git-based version control at all transformation levels. Each major milestone including initial requirement structuring, model synthesis, validation, verification, and code generation is committed as a tagged release (e.g., `v0.1-initial-requirements`, `v0.3-validated-aadl`, `v0.5-verified-code`). This tracking simplifies audit and certification processes while facilitating rollback and root-cause analysis during iterative development. AADL-aware diff tools enable visualization and comparison of model differences, minimizing complexity when managing large architectures.

4.7 Deployment and Code Generation

Following successful verification, the framework facilitates automated generation of executable code skeletons based on target application domains. Code generation targets include C, C++, and ROS2, depending on platform requirements. Structural and interface consistency are preserved during code skeleton generation from verified AADL models. While generated skeletons lack dense application logic such as device drivers and control algorithms, they provide stable foundations reducing implementation time and preserving architectural coherence.

The approach is inherently modular and scalable, designed to support high-assurance application domains including industrial automation, automotive, and avionics. Adaptation to other domains such as smart infrastructure or medical devices is achieved by updating contract libraries, validation schemata, and fine-tuning LLMs with domain-specific data. By decomposing systems into manageable subsystems that are independently modelable, verifiable, and synthesizable, the approach enables parallel development and distributed verification for large-scale CPS.

5 CASE STUDIES

To validate the effectiveness and practical viability of our work, we conducted two case studies targeting representative cyber-physical systems from distinct application domains. These case studies evaluate core attributes including syntactic accuracy, semantic fidelity, development efficiency, adaptability across domains, and compliance with safety-critical standards.

5.1 Case Study 1: Unmanned Aerial Vehicle System

5.1.1 System Overview. The Unmanned Aerial Vehicle system is designed with a complex, modular architecture using AADL, consisting of multiple interconnected subsystems. The system specification outlines a robust design with distinct component responsibilities and performance characteristics, ensuring reliable and efficient operation. The UAV system comprises the following subsystems:

• Flight Control System: Manages real-time flight dynamics and control surface actuation • Sensor Array: Integrates multiple sensor types for environmental awareness • Communication Module: Handles telemetry transmission and command reception • Power Management: Ensures optimal energy distribution across subsystems • Navigation Processor: Performs trajectory calculation and spatial reasoning • Error Handling Mechanism: Implements fault detection and recovery procedures

5.1.2 Performance Characteristics and System Dynamics. The UAV system's architectural performance is characterized by carefully allocated computing resources and data management strategies. The flight control processor, operating at 1.2 GHz with 8 GB of memory, represents a critical computational node designed to manage real-time sensor processing and vehicle control algorithms. The navigation processor demonstrates enhanced computational capacity, featuring a 1.5 GHz processor and 16 GB memory allocation, enabling complex trajectory calculations and advanced spatial reasoning.

The system's communication architecture implements a multi-layered data flow method ensuring robust information transmission. Sensor data undergoes systematic routing through the flight computer, facilitating comprehensive environmental interpretation and decision-making. Simultaneously, telemetry transmission occurs through a dedicated communication module, implementing encrypted protocols to maintain system integrity and security. Power management strategies are intricately integrated across all components, ensuring uniform energy distribution and optimizing overall system reliability.

Operational resilience is achieved through a carefully designed three-mode error handling framework. The system transitions between normal operational parameters, emergency response protocols, and autonomous landing methods. This adaptive structure enables dynamic response to varying environmental conditions and potential system anomalies.

5.1.3 AADL Model Generation Results. The framework successfully generated comprehensive AADL models for the UAV system from natural language requirements. The following excerpt demonstrates the automatically generated system implementation:

Listing 1: UAV System AADL Implementation

```
system implementation UAV.standard
   subcomponents
      flight_computer: processor Flight_Control_Processor {
         Scheduling_Protocol => RMS;
      };

      sensor_subsystem: system Sensor_Array;

      communication_module: device Communication_Unit {
         Communication_Properties => {
            Transmission_Type => Wireless;
            Frequency_Range => 2.4 GHz .. 5.8 GHz;
         };
      };

      power_system: system Power_Management;
      nav_processor: processor Navigation_Processor;
      error_handler: process Error_Handling_System;

   connections
      sensor_to_flight: port sensor_subsystem.sensor_data →
                              flight_computer.sensor_input;
      flight_to_comm: port flight_computer.telemetry_output →
                           communication_module.tx_data;
      nav_to_flight: port nav_processor.trajectory_data →
                          flight_computer.nav_input;
end UAV.standard;
```

The Flight Control Processor specification illustrates detailed component modeling:

Listing 2: Flight Control Processor AADL Specification

```
processor Flight_Control_Processor
   features
      sensor_input: in data port;
      nav_input: in data port;
      control_output: out data port;
      telemetry_output: out data port;
   properties
      Processor_Speed => 1.2 GHz;
      RAM_Size => 8 GB;
      Scheduling_Protocol => RMS;
      Clock_Period => 1 ms;
      Execution_Time => 0.5 ms .. 0.8 ms;
end Flight_Control_Processor;
```

5.1.4 Code Generation and Verification. Following AADL model validation, the framework generated implementation code for critical UAV components. The error correction module successfully identified and resolved several issues including timing constraint violations in sensor processing threads and resource allocation conflicts between navigation and flight control processors.

5.2 Case Study 2: Smart Building System

5.2.1 System Overview. The second case study focuses on an integrated smart building system combining HVAC, lighting, and security subsystems. The objective was to achieve unified optimization maintaining occupant safety, comfort, and energy efficiency across multiple domains. This case study demonstrates the framework's capability in managing complex multi-domain CPS with diverse requirements and constraints.

5.2.2 Multi-Zone HVAC System. The multi-zone HVAC system regulates temperature across multiple rooms, each with independent sensors and actuators including heaters and air conditioning units. The system must maintain comfortable temperature ranges (18°C-25°C) while optimizing energy consumption.

The framework automatically generated the following controller implementation:

Listing 3: Temperature Controller Implementation

```
class ZoneSensor:
    def __init__(self, zone_id):
        self.zone_id = zone_id
        self.current_temperature = 22.0

    def read_temperature(self):
        # Integration with hardware sensor interface
        return self.current_temperature

class TemperatureController:
    def __init__(self, zone_sensor, heater, ac_unit,
                 target_temperature=22.0):
        self.zone_sensor = zone_sensor
        self.heater = heater
        self.ac_unit = ac_unit
        self.target_temperature = target_temperature
        self.hysteresis = 2.0  # Prevent rapid switching

    def regulate_temperature(self):
        current_temp = self.zone_sensor.read_temperature()
        if current_temp < self.target_temperature - self.hysteresis:
            self.heater.turn_on()
            self.ac_unit.turn_off()
        elif current_temp > self.target_temperature +
      self.hysteresis:
            self.ac_unit.turn_on()
            self.heater.turn_off()
        else:
            self.heater.turn_off()
            self.ac_unit.turn_off()
```

5.2.3 Error Correction Example. Initial testing revealed that the energy optimizer did not account for external weather conditions, leading to suboptimal performance. The error correction module, using CoT reasoning, identified this limitation and refined the implementation:

Listing 4: Refined Energy Optimizer

```
class EnergyOptimizer:
    def __init__(self, external_weather_service):
        self.external_weather = external_weather_service

    def optimize(self, zones):
        current_weather = self.external_weather.get_conditions()
        for zone in zones:
            # Adjust target temperatures based on weather
            if current_weather == "Cold":
                zone.target_temperature += 1.0
            elif current_weather == "Hot":
                zone.target_temperature -= 1.0

            # Additional optimization for predictive control
            if current_weather == "Transition":
                zone.enable_predictive_mode()
```

5.2.4 Smart Lighting System. The smart lighting system dynamically adjusts illumination levels based on occupancy and ambient light, ensuring sufficient illumination (minimum 300 lux) while minimizing energy consumption:

Listing 5: Light Controller with Emergency Handling

```
class LightController:
    def __init__(self, occupancy_sensor, ambient_light_sensor,
                 emergency_alarm):
        self.occupancy_sensor = occupancy_sensor
        self.ambient_light_sensor = ambient_light_sensor
        self.emergency_alarm = emergency_alarm
        self.min_lux = 300

    def adjust_lighting(self):
        # Safety-critical: Emergency override
        if self.emergency_alarm.is_triggered():
            self.turn_on_lights()
            return

        # Normal operation
        occupied = self.occupancy_sensor.is_occupied()
        ambient = self.ambient_light_sensor.read_light()

        if occupied and ambient < 50:
            self.turn_on_lights()
        elif not occupied:
            self.turn_off_lights()
        else:
            self.dim_lights()
```

During testing, the error correction module identified the lack of emergency handling and automatically refined the implementation to prioritize safety in critical situations.

5.2.5 Integrated System Controller. The framework synthesized an overarching control strategy coordinating all subsystems:

Listing 6: Integrated Building Controller

```
class IntegratedController:
    def __init__(self, hvac_controller, light_controller,
                 security_system, energy_optimizer):
        self.hvac_controller = hvac_controller
        self.light_controller = light_controller
```

Table 1: Evaluations of LLMs in Integrated CPS Development

LLM	Syntax Acc. (%)	Fidelity (%)	Error (%)	Speed (ms)	Complexity
Llama3	88	83	8	94	Moderate
Claude3	85	88	11	130	High
ChatGPT4	82	81	17	110	High

```
        self.security_system = security_system
        self.energy_optimizer = energy_optimizer

    def operate(self):
        # Safety-first approach
        if self.security_system.is_emergency():
            self.hvac_controller.turn_off()
            self.light_controller.turn_on_lights()
            self.security_system.activate_alarm()
        else:
            # Normal optimization
            self.energy_optimizer.optimize()
            self.hvac_controller.regulate_temperature()
            self.light_controller.adjust_lighting()
```

5.3 Quantitative Evaluation

We evaluated three LLMs, Llama3, Claude3, and ChatGPT4, to assess their impact on CPS code generation Table 1. Llama3 achieved the highest syntax accuracy (88%) and lowest error rate (8%), while Claude3 showed semantic comprehension with the highest requirement fidelity (88%). ChatGPT4 offered competitive execution speed but exhibited higher overall errors, showing model-specific tradeoffs. Automated model synthesis reduced development time by approximately 70%, produced models exceeding 80% syntax accuracy, and improved traceability, ensuring all requirements were captured. Formal verification using AGREE confirmed that over 90% of generated models satisfied system-level properties on the first attempt. Domain expert involvement was largely limited to resolving minor ambiguities, reducing manual effort by 60%. The framework scaled effectively to complex scenarios, managing HVAC systems with 50+ zones, lighting for 100+ rooms, and integrated building systems with over 200 sensors and actuators, highlighting robust efficiency, reliability, and energy savings of 15–25% for large-scale CPS deployments.

5.4 Domain Adaptability

To assess cross-domain applicability, the framework was tested against simplified CPS examples from automotive and avionics domains. With minor adjustments to LLM prompt templates and validation schemas, the system successfully generated AADL models aligned with safety assurance levels including DAL B (Design Assurance Level) for avionics and ASIL C (Automotive Safety Integrity Level) for automotive applications. These results confirm the framework's adaptability across multiple CPS domains.

6 DISCUSSION

The proposed work shows strong potential for automating cyber-physical systems development. The integration of formal methods such as bounded model checking, timed automata verification, and contract-based reasoning provides robust assurance of safety-critical properties, with generated artifacts conforming to industry standards (DAL B, ASIL C). Successful application across aerospace and smart infrastructure domains further demonstrates the framework's practical utility and cross-domain adaptability.

Despite these strengths, some challenges remain. Framework performance is highly dependent on the quality and completeness of input requirements, as ambiguous or inconsistent specifications lead to suboptimal model generation. Although interactive feedback mitigates this issue, human intervention remains necessary, partially offsetting automation gains. In addition, while core AADL constructs are well supported, comprehensive automation of annex languages (e.g., timing, error, and behavioral annexes) requires further tooling. Computational overhead associated with LLM-based generation and correction may also limit applicability in latency-sensitive contexts, motivating future work on optimization techniques such as prompt caching and incremental generation.

Scalability represents another major consideration. While the framework scales to moderately complex systems, validation on very large-scale CPS remains open, particularly given LLM context limitations and state explosion in formal verification. Addressing these issues will require hierarchical decomposition, compositional verification, and efficient knowledge management. Finally, the framework adopts a human–AI collaboration paradigm in which AI augments expert decision-making rather than replacing it. Optimizing task allocation, explainability, and trust calibration remains an important direction for future research to ensure effective industrial adoption.

7 CONCLUSION

This paper presents an integrated framework that addresses fundamental challenges in cyber-physical systems development through a mutual combination of AADL-based model-driven engineering and LLM-based code generation. The proposed framework shows that domain-adapted LLMs can automate large portions of CPS development while preserving compliance with safety-critical requirements. By integrating formal verification techniques at both the model and code levels, incorporating intelligent CoT–based error correction designed to CPS constraints, and using a dual-loop feedback method for incremental refinement and cyclic improvement, the framework mitigates key limitations of existing approaches, including high manual effort, difficulties in maintaining consistency, and limited adaptability. This work establishes a foundation for next-generation CPS development tools, showing that intelligent automation can accelerate development while upholding the stringent safety and reliability standards required for critical applications in aerospace, smart infrastructure, and related domains.

References

[1] Md Al Maruf, Akramul Azim, Nitin Auluck, and Mansi Sahi. 2024. FeaMod: Enhancing Modularity, Adaptability and Code Reuse in Embedded Software Development. In *2024 IEEE International Conference on Information Reuse and Integration for Data Science (IRI)*. IEEE, 246–251.

[2] Rohan Anil, Andrew M Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403* (2023).

[3] Radhakisan Baheti and Helen Gill. 2011. Cyber-physical systems. *The impact of control technology* 12, 1 (2011), 161–166.

[4] Thomas Ball, Abhijith Chatra, Peli de Halleux, Steve Hodges, Michał Moskal, and Jacqueline Russell. 2019. Microsoft MakeCode: embedded programming for education, in blocks and TypeScript. In *Proceedings of the 2019 ACM SIGPLAN symposium on SPLASH-E*. 7–12.

[5] Tom B Brown. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* (2020).

[6] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. 2023. Sparks of artificial general intelligence: Early experiments with GPT-4. arXiv. *arXiv preprint arXiv:2303.12712* (2023).

[7] Thidapat Chantem, Nan Guan, and Duo Liu. 2019. Sustainable embedded software and systems. 152–154 pages.

[8] Angelica Chen, Jérémy Scheurer, Tomasz Korbak, Jon Ander Campos, Jun Shern Chan, Samuel R Bowman, Kyunghyun Cho, and Ethan Perez. 2023. Improving code generation by training with natural language feedback. *arXiv preprint arXiv:2303.16749* (2023).

[9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[10] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128* (2023).

[11] Marco Conti, Sajal K Das, Chatschik Bisdikian, Mohan Kumar, Lionel M Ni, Andrea Passarella, George Roussos, Gerhard Tröster, Gene Tsudik, and Franco Zambonelli. 2012. Looking ahead in pervasive computing: Challenges and opportunities in the era of cyber–physical convergence. *Pervasive and mobile computing* 8, 1 (2012), 2–21.

[12] Rogério De Lemos, Holger Giese, Hausi A Müller, Mary Shaw, Jesper Andersson, Marin Litoiu, Bradley Schmerl, Gabriel Tamura, Norha M Villegas, Thomas Vogel, et al. 2013. Software engineering for self-adaptive systems: A second research roadmap. In *Software Engineering for Self-Adaptive Systems II: International Seminar, Dagstuhl Castle, Germany, October 24-29, 2010 Revised Selected and Invited Papers*. Springer, 1–32.

[13] Nilanjan Dey, Amira S Ashour, Fuqian Shi, Simon James Fong, and João Manuel RS Tavares. 2018. Medical cyber-physical systems: A survey. *Journal of medical systems* 42 (2018), 1–13.

[14] Okwudili M. Ezeme, Qusay H. Mahmoud, and Akramul Azim. 2022. A Framework for Anomaly Detection in Time-Driven and Event-Driven Processes Using Kernel Traces. *IEEE Transactions on Knowledge and Data Engineering* 34, 1 (2022), 1–14. doi:10.1109/TKDE.2020.2978469

[15] Jinghua Guo, Lubin Li, Jingyao Wang, and Keqiang Li. 2022. Cyber-physical system-based path tracking control of autonomous vehicles under cyber-attacks. *IEEE Transactions on Industrial Informatics* 19, 5 (2022), 6624–6635.

[16] Zhi Guo, Walid Najjar, and Betul Buyukkurt. 2008. Efficient hardware code generation for FPGAs. *ACM Transactions on Architecture and Code Optimization (TACO)* 5, 1 (2008), 1–26.

[17] Imre Horvath and Bart HM Gerritsen. 2012. Cyber-physical systems: Concepts, technologies and implementation principles. In *Proceedings of TMCE*, Vol. 1. 7–11.

[18] Geunwoo Kim, Pierre Baldi, and Stephen McAleer. 2024. Language models can solve computer tasks. *Advances in Neural Information Processing Systems* 36 (2024).

[19] Heiko Koziolek. 2011. Sustainability evaluation of software architectures: a systematic review. In *Proceedings of the joint ACM SIGSOFT conference–QoSA and ACM SIGSOFT symposium–ISARCS on Quality of software architectures–QoSA and architecting critical systems–ISARCS*. 3–12.

[20] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2024. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36 (2024).

[21] Mingjie Liu, Teodor-Dumitru Ene, Robert Kirby, Chris Cheng, Nathaniel Pinckney, Rongjian Liang, Jonah Alben, Himyanshu Anand, Sanmitra Banerjee, Ismet Bayraktaroglu, et al. 2023. Chipnemo: Domain-adapted llms for chip design. *arXiv preprint arXiv:2311.00176* (2023).

[22] Xin Liu, Daniel McDuff, Geza Kovacs, Isaac Galatzer-Levy, Jacob Sunshine, Jiening Zhan, Ming-Zher Poh, Shun Liao, Paolo Di Achille, and Shwetak Patel. 2023. Large language models are few-shot health learners. *arXiv preprint arXiv:2305.15525* (2023).

[23] Zhiqiang Liu, Yong Dou, Jingfei Jiang, and Jinwei Xu. 2016. Automatic code generation of convolutional neural networks in FPGA implementation. In *2016 International conference on field-programmable technology (FPT)*. IEEE, 61–68.

[24] Md Al Maruf and Akramul Azim. 2020. Extending resources for avoiding overloads of mixed-criticality tasks in cyber-physical systems. *IET Cyber-Physical Systems: Theory & Applications* 5, 1 (2020), 60–70.

[25] Peter Marwedel. 2021. *Embedded system design: embedded systems foundations of cyber-physical systems, and the internet of things*. Springer Nature.

[26] Henry Muccini, Mohammad Sharaf, and Danny Weyns. 2016. Self-adaptation for cyber-physical systems: a systematic literature review. In *Proceedings of the 11th international symposium on software engineering for adaptive and self-managing systems*. 75–81.

[27] National Science Foundation. 2010. Cyber-Physical Systems (CPS) Program Solicitation. https://new.nsf.gov/funding/opportunities/cps-cyber-physical-systems/nsf10-515/solicitation Accessed: 2024-10-11.

[28] Nhan Nguyen and Sarah Nadi. 2022. An empirical evaluation of GitHub copilot's code suggestions. In *Proceedings of the 19th International Conference on Mining Software Repositories*. 1–5.

[29] Erik Nijkamp, Hiroaki Hayashi, Caiming Xiong, Silvio Savarese, and Yingbo Zhou. 2023. Codegen2: Lessons for training llms on programming and natural languages. *arXiv preprint arXiv:2305.02309* (2023).

[30] Sudeep Pasricha. 2022. Embedded systems education in the 2020s: Challenges, reflections, and future directions. In *Proceedings of the Great Lakes Symposium on VLSI 2022*. 519–524.

[31] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2023. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334* (2023).

[32] Gabriel Poesia, Oleksandr Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. 2022. Synchromesh: Reliable code generation from pre-trained language models. *arXiv preprint arXiv:2201.11227* (2022).

[33] Luisa Restrepo, Jose Aguilar, Mauricio Toro, and Elizabeth Suescun. 2021. A sustainable-development approach for self-adaptive cyber–physical system's life cycle: A systematic mapping study. *Journal of Systems and Software* 180 (2021), 111010.

[34] Luis Ribeiro and Mats Björkman. 2017. Transitioning from standard automation solutions to cyber-physical production systems: an assessment of critical conceptual and technical challenges. *IEEE systems journal* 12, 4 (2017), 3816–3827.

[35] Matthias C Rillig, Marlene Ågerstrand, Mohan Bi, Kenneth A Gould, and Uli Sauerland. 2023. Risks and benefits of large language models for the environment. *Environmental Science & Technology* 57, 9 (2023), 3464–3466.

[36] Darren Saguil and Akramul Azim. 2020. A layer-partitioning approach for faster execution of neural network-based embedded applications in edge networks. *IEEE Access* 8 (2020), 59456–59469.

[37] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2024. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2024).

[38] John A Stankovic. 2014. Research directions for the internet of things. *IEEE internet of things journal* 1, 1 (2014), 3–9.

[39] A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).

[40] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171* (2022).

[41] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.

[42] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C Schmidt. 2023. A prompt pattern catalog to enhance prompt engineering with chatgpt. *arXiv preprint arXiv:2302.11382* (2023).

[43] Xin Yin, Chao Ni, Shaohua Wang, Zhenhao Li, Limin Zeng, and Xiaohu Yang. 2024. Thinkrepair: Self-directed automated program repair. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1274–1286.

[44] Xinghuo Yu and Yusheng Xue. 2016. Smart grids: A cyber–physical systems perspective. *Proc. IEEE* 104, 5 (2016), 1058–1070.

[45] Sherali Zeadally, Teodora Sanislav, and George Dan Mois. 2019. Self-adaptation techniques in cyber-physical systems (CPSs). *IEEE Access* 7 (2019), 171126–171139.

[46] Tehseen Zia, Abdul Wahab, David Windridge, Santosh Tirunagari, and Nauman Bashir Bhatti. 2023. Visual attribution using adversarial latent transformations. *Computers in Biology and Medicine* 166 (2023), 107521.